*Loose coupling* is an essential property of a system where parts have as little knowledge of each other as possible. The goal is to evolve each piece independently so that when one is changed, the others don't need to change accordingly.

*Coupling*, and its twin concept *cohesion*, have played an essential role in software engineering for decades. It's a good design practice to decompose a system into modules (*modularization*), each of which has minimal dependencies on the other parts (loose coupling) and to encapsulate code that changes together (high cohesion). Depending on the architectural style, a module can model a monolithic component or a standalone service (for example, a microservice). Either way, we should aim at achieving proper modularization with loose coupling and high cohesion.

Parnas identified three benefits of modularization:[6]

- *Managerial*—Since each module is loosely coupled, the team responsible for it should not need to spend much time coordinating and communicating with other teams.
- *Product flexibility*—The overall system should be flexible since each module is evolved independently of the others.
- *Comprehensibility*—People should be able to understand and work with a module without having to study the whole system.

The preceding benefits are usually among those associated with microservices, but the truth is that you don't need microservices to achieve them. In the last few years, many organizations have decided to migrate from monoliths to microservices. Some of them have failed because they lacked proper modularization. A monolith made up of tightly coupled, non-cohesive components, when migrated, produces a tightly coupled, non-cohesive microservice system, which sometimes is referred to as a